

SILAB[®]

Version 7.1
Referenz Ruby Scripting

INHALTSVERZEICHNIS

1	ÜBERBLICK.....	3
2	GRUNDLEGENDE HILFSFUNKTIONEN.....	3
3	ERSTELLEN EINES RUBY-SKRIPTS (RPU).....	4
3.1	Anmerkungen zur Deklaration von Ein- und Ausgangsvariablen	6
3.2	Behandlung weiterer Simulations-Ereignisse	7
3.3	Weitere Informationen zu RPU's	8
4	INFORMATIONEN ZUR AKTIVEN SIMULATION.....	8
5	ZUGRIFF AUF DIE OBJEKTDATENBANK ODB	8
6	ZUGRIFF AUF DIE DATENBASIS SCNX	9

1 ÜBERBLICK

Ruby ist eine objektorientierte Programmiersprache, die umfangreiche Funktionalität in einer kompakten und übersichtlichen Syntax bietet. Ausgiebiges Lehrmaterial findet sich auf der Homepage <http://www.ruby-lang.org/> und weiterhin unter <http://www.ruby-doc.org/>. Besonders hilfreich wird sicher das Buch „Programming Ruby“ sein, welches online unter <http://www.ruby-doc.org/docs/ProgrammingRuby/> gelesen werden kann.

Eine Referenz der Standard-Ruby-Klassen und -Module, die in den Skripten verfügbar sind, kann unter <http://ruby-doc.org/core-2.3.0/> abgerufen werden.

Mit der **DPURubyScript** bietet SILAB eine Integration eines Ruby-Interpreters, die es ermöglicht Ruby-Skripte genauso wie im Lieferumfang vorhandene DPUs zu verwenden, um leicht während der Simulation Berechnungen auszuführen. Diese Ruby-Skripte (im Weiteren *Ruby Processing Unit* oder kurz *RPU* genannt) können beliebige Ein- und Ausgangsvariablen definieren und darüber mit der Simulation kommunizieren.

Hinweis: Der Ruby-Interpreter in SILAB enthält nur die *Ruby Core API*, nicht die *Ruby Std-lib*.

Die Erstellung eines einfachen Ruby-Skripts und seine Integration in die Simulation wird in Kap. 3 dieses Dokuments beschrieben. Für die Integration wird neben der *Ruby Core API* auch eine SILAB-spezifische *SILAB Ruby API* verwendet. Eine detaillierte Dokumentation dieser API finden Sie unter <D:\SILAB\doc\ruby\index.html>.

Falls auf Ihrem Simulator *SILAB enterprise* mit dem *SCN-* bzw. *ODB-Entwicklerpaket* installiert ist, sind zusätzliche Klassen verfügbar, die einen direkten Zugriff auf die Datenbasis SCN bzw. die Objektdatenbank ODB ermöglichen. Hinweise hierzu finden Sie in Kap. 5 und 6.

2 GRUNDLEGENDE HILFSFUNKTIONEN

Überall in RPU-Skripten sind Funktionen zur Ausgabe von Meldungen in die SILAB-Logdateien verfügbar. Folgende Funktionen sind hierfür definiert:

Funktionsname	Ziel der Meldung
<i>log_debug(anObject, ...)</i>	Logdatei „SILAB.debug.log“
<i>log_err(anObject, ...)</i>	Logdatei „SILAB.err.log“
<i>log_sys_info(obj, ...)</i>	Logdatei „SILAB.sys.log“ und der SILAB-Meldungsbereich ⓘ
<i>log_sys_warning(obj, ...)</i>	Logdatei „SILAB.sys.log“ und der SILAB-

log_sys_error(obj, ...)

Meldungsbereich (📢)
Logdatei „SILAB.sys.log“ und der SILAB-Meldungsbereich (📢)

Weitere Informationen hierzu finden Sie in der *SILAB Ruby API* Dokumentation, im Modul *Kernel*.

3 ERSTELLEN EINES RUBY-SKRIPTS (RPU)

Um Berechnungen während der Laufzeit der Simulation auszuführen, können Sie eine Ruby Programming Unit (RPU) erstellen.

- Das folgende Beispiel erstellt die **RPUMittelwert** zur Berechnung eines gleitenden Mittels über 3 Simulationsschritte.
- Erstellen Sie eine Textdatei mit dem Namen `RPUMittelwert.rb` und speichern Sie diese im Unterverzeichnis `D:\SILAB\MyProject\scripts` ab. Dazu können Sie einen beliebigen Texteditor (z.B. Notepad) verwenden. Bessere Editoren wie Geany oder Notepad++ sind sehr empfehlenswert, da sie die Erstellung durch eine automatische Syntaxhervorhebung vereinfachen.
- Definieren Sie in der Textdatei eine Ruby-Klasse mit dem Namen `RPUMittelwert` (der Name muss gleich sein wie der Dateiname ohne Erweiterung) und geben Sie an, dass diese Klasse die Basisklasse `RPU` aus der *SILAB Ruby API* erweitert.

```
class RPUMittelwert < RPU

  def initialize()
  end
end
```

- Während des Simulationsablaufs werden durch *SILAB* verschiedene Methoden Ihrer `RPU`-Klasse aufgerufen. Beispielsweise wird beim Laden der Versuchskonfiguration einmalig die Methode `initialize()` Ihrer `RPU`-Klasse aufgerufen.
- In der `initialize()` Methode können die Ein- und Ausgangsvariablen für Ihre `RPU` definiert werden. Dazu rufen Sie die `var_in`-Methode bzw. `var_out` Methode für jede Variable auf. Als Parameter wird ein Symbolname einer Instanzvariablen Ihrer `RPU`-Klasse erwartet, d.h. `:@my_var_name` (d.h. der Variablenname wird mit dem Präfix `:` angegeben).
- In Ruby müssen Variablennamen mit Kleinbuchstaben beginnen und Wortteile werden durch Unterstriche getrennt. In *SILAB* beginnen Variablennamen und alle ihre Wortteile hingegen üblicherweise mit einem Großbuchstaben und die

Worteile werden ohne Unterstriche zusammengefügt. Dies wird bei der Angabe von Variablennamen in `var_in` bzw. `var_out` automatisch berücksichtigt, d.h. Unterstriche werden entfernt und der Anfangsbuchstabe jedes Wortteils wird großgeschrieben.

- Passen Sie die `initialize`-Methode des Beispiels also wie folgt an, um die RPU um einen Eingang *Input* und einen Ausgang *Output* zu ergänzen:

```
def initialize()
  var_in  :@input  # DPU-Eingang 'Input' anlegen
  var_out :@output # DPU-Ausgang 'Output' anlegen
end
```

- Fügen Sie nun zusätzlich eine `start`-Methode in die RPU-Klasse ein, um vor dem Start der Simulation jeweils den internen Zustand der RPU zu initialisieren:

```
def start()
  # Zwei Variablen zur Zwischenspeicherung der letzten beiden
  # Eingangswerte anlegen.
  @in2 = 0.0
  @in3 = 0.0
  # ReadyToStart erfolgreich
  return RPU_READY
end
```

- Die eigentliche Berechnung erfolgt in der `trigger`-Methode, die einmal in jedem Simulationsschritt aufgerufen wird:

```
def trigger(time, time_error)
  # Ausgabewert berechnen (Mittelwert des aktuellen und der
  # zwei gespeicherten Eingangswerte).
  @output = (@input + @in2 + @in3) / 3.0

  # Gespeicherte Werte weiterschieben
  @in3 = @in2
  @in2 = @input
end
```

- Die Funktionalität der `RPUMittelwert` ist nun vollständig implementiert. Sie kann nun über die `DPURubyScript` in die Simulation integriert werden. Ein vollständiges Beispiel für eine passende Konfigurationsdatei ist:

```
SILAB Configuration
{
  Computerconfiguration Rechnernetze
  {
    Pool Einzelplatz
    {
      Executeable = true;

      Computer LOCALHOST
      {
        IP = "127.0.0.1";
        PeriodMS = 100;
        Operator = true;
      }
    }
  }
}
```

```

    };

    };
};

DPUConfiguration DPUs
{
    Pool Aufzeichnung
    {
        Executeable = true;

        DPUClock Clock
        {
            Computer = {LOCALHOST};
            Index = 0;
        };

        DPUOscillator Speed
        {
            Computer = {LOCALHOST};
            Start = 1;
            Period = 2000;
            On = 1;
            Index = 0;
            Amplitude = 125;
            Offset = 125;
        };

        DPURubyScript Mittelwert
        {
            Index = 1;
            Computer = {LOCALHOST};

            Script = "RPUMittelwert.rb";    #Hier wird das Ruby-Skript
                                           #eingebunden. Dieses definiert
                                           #die Ein- und Ausgänge der
                                           #DPURubyScript-Instanz Mittelwert

        };

        Connections =
        {
            Speed.Out <-> Mittelwert.Input
        };
    };
};
};
};

```

3.1 Anmerkungen zur Deklaration von Ein- und Ausgangsvariablen

Alternativ zur einfachen Definition von Ein- und Ausgangsvariablen mit den Methoden `var_in` und `var_out` können auch erweiterte Eigenschaften für die Variablen festgelegt werden. Dazu wird dann die Methode `var_in_with` bzw. `var_out_with` verwendet.

Hierbei können zusätzliche Parameter übergeben werden:

Parameter	Beschreibung
-----------	--------------

type	Wird der Wert <code>VT_STRING</code> angegeben, wird die Variable in SILAB als Textvariable registriert. (Standardmäßig wird eine numerische Variable erstellt.)
name	Optional kann ein abweichender Name für die Variable in SILAB angegeben werden.
def	Für numerische Variablen kann ein Standardwert festgelegt werden, den diese Variable immer beim Laden der Simulation annimmt, wenn kein anderer angegeben wurde.
min	Für numerische Variablen kann ein Minimalwert festgelegt werden. Während der Simulation kann die Variable diesen Wert nicht unterschreiten.
max	Für numerische Variablen kann ein Maximalwert festgelegt werden. Während der Simulation kann die Variable diesen Wert nicht überschreiten.

Einige Beispiele hierfür:

Aufruf im Ruby-Skript	Beschreibung der Variablen in SILAB
<code>var_in :@x</code>	Numerische Eingangsvariable "x" mit Standardwert 0. Sie kann beliebige Werte annehmen.
<code>var_in_with :@text, VT_STRING, "Message"</code>	Eingangsvariable "Message", in die beliebige Zeichenketten eingegeben werden können.
<code>var_in_with :@vehicle_length, 4.73, 0.5, 1000.0</code>	Eingangsvariable "VehicleLength" mit dem Standardwert 4.73. Der Wertebereich der Variablen wird auf den Bereich 0.5 bis 1000 beschränkt.

3.2 Behandlung weiterer Simulations-Ereignisse

In Ihrer RPU-Klasse können Sie auf weitere Simulations-Ereignisse reagieren. Indem Sie folgende optionale Methoden hinzufügen, können Sie an den entsprechenden Stellen des Simulationsablaufs weitere Aktionen durchführen.

Methodenname	Beschreibung
<code>prepare() →</code>	Diese Methode wird beim Laden der SILAB-

<i>true/false</i>	Versuchskonfiguration einmalig aufgerufen. Sie sollte <code>true</code> zurückgeben, wenn die Vorbereitung erfolgreich war, oder <code>false</code> , wenn ein Fehler auftrat. In diesem Fall wird das Laden der Konfiguration abgebrochen.
<i>start() → RPU_XXX</i>	Diese Methode wird beim Launch der SILAB-Versuchskonfiguration aufgerufen. Sie sollte <code>RPU_READY</code> zurückgeben, wenn die Vorbereitung erfolgreich war, oder <code>RPU_ABORT</code> , wenn ein Fehler auftrat. Der Launch der Konfiguration wird dann abgebrochen.
<i>trigger(t, te)</i>	Diese Methode wird in jedem Simulationsschritt einmal aufgerufen. Als Parameter werden zwei Gleitkommazahlen (Ruby <code>Float</code> -Objekte) übergeben: <i>t</i> Zeit seit dem Start der Simulation (in Millisekunden) <i>te</i> Aktuelle Rechenzeitverzögerung (in Millisekunden)
<i>stop() → RPU_XXX</i>	Diese Methode wird beim Stoppen der Simulation aufgerufen. Die Methode sollte stets <code>RPU_READY</code> zurückgeben.
<i>release()</i>	Diese Methode wird beim Herunterfahren der Simulation („Shutdown“ bzw. Beenden von SILAB) aufgerufen. Hier sollten alle von der RPU belegten Ressourcen freigegeben werden.

3.3 Weitere Informationen zu RPUs

Weitere Informationen zur RPU-Klasse finden Sie in der SILAB Ruby API-Dokumentation unter <D:\SILAB\doc\ruby\RPU.html>.

4 INFORMATIONEN ZUR AKTIVEN SIMULATION

Das Modul SILAB enthält Funktionen, um globale Informationen zur aktiven Simulation zu erhalten. Mit den darin enthaltenen Funktionen können beispielsweise Dateien im SILAB-Verzeichnis gesucht werden. Weitere Informationen hierzu finden Sie in der SILAB Ruby API-Dokumentation unter <D:\SILAB\doc\ruby\SILAB.html>.

5 ZUGRIFF AUF DIE OBJEKTDATENBANK ODB

Falls auf Ihrem Simulator *SILAB enterprise* mit dem *ODB-Entwicklerpaket* installiert ist, sind die zusätzlichen Klassen **ODBQuery** und **ODBObject** verfügbar, mit denen Anfragen an die Objektdatenbank ausgeführt und ausgewertet werden können.

Beachten Sie, dass in der SILAB-DPU-Konfiguration folgendes eingetragen werden muss, damit alle Objekte in ODB bereitgestellt werden:

```
DPUPool X {  
  ...  
  SCN.ODB = true;  
  TRF.ODB = true;  
  MOV.ODB = true;  
};
```

Da die Abfragen sehr den entsprechenden Klassen in C++ ähneln, werden hier keine Details erläutert. Lesen Sie daher bitte auch das Dokument „Referenz Objektdatenbank ODB“, um mehr über die Funktionsweise von ODB zu erfahren.

Details zu den Klassen ODBQuery und ODBObject finden Sie in der SILAB Ruby API-Dokumentation unter <D:\SILAB\doc\ruby\ODBQuery.html>.

6 ZUGRIFF AUF DIE DATENBASIS SCNX

Falls auf Ihrem Simulator *SILAB enterprise* mit dem *ODB-Entwicklerpaket* installiert ist, sind zusätzliche Klassen verfügbar, um Cursor in der Datenbasis SCNX aufzusetzen (*SCNCursor*) und so Informationen über die Straßengeometrie zu erhalten. Außerdem besteht die Möglichkeit, Informationen über die Straße an der Position des Egofahrzeugs abzufragen (*SCNManagedCursor*) und auf Strecken-Events zu reagieren (*SCNHedgehog*).

Details zu den SCN-bezogenen Klassen finden Sie in der SILAB Ruby API-Dokumentation unter <D:\SILAB\doc\ruby\index.html>.